

Section 3

2026年1月26日 星期一 14:50

3 Data flow from register file to ALU

1. Register file to ALU connections. Using wire tracing in ISSIE, identify through which ports register data is sent from the register file to the ALU.

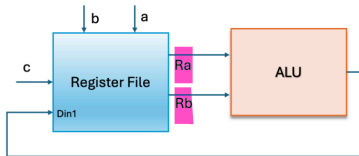
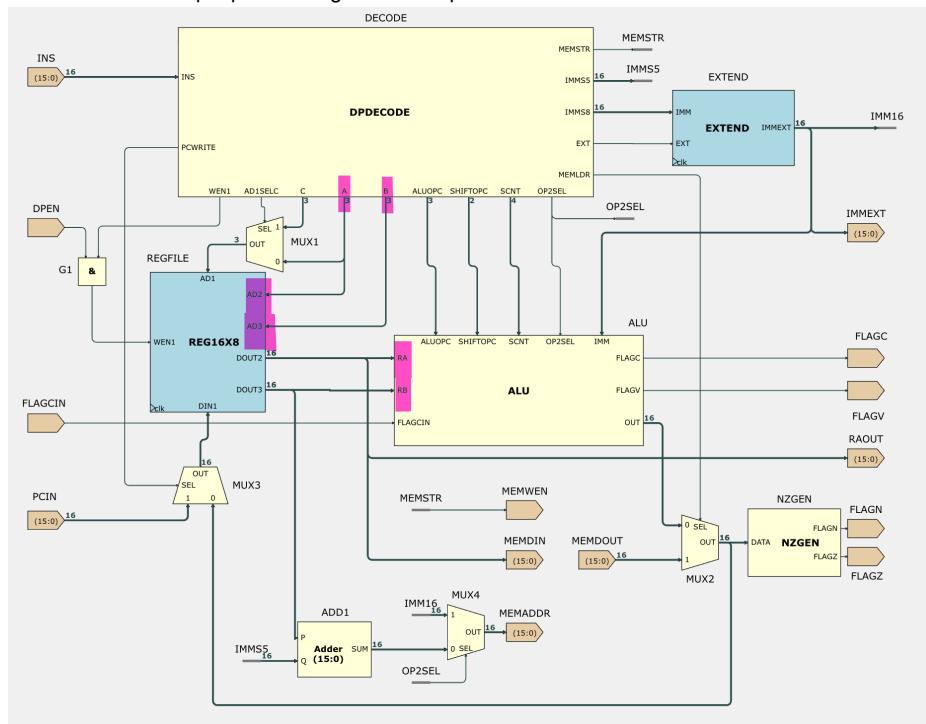


Figure 4: EEP1 datapath simplified

Ra & Rb.

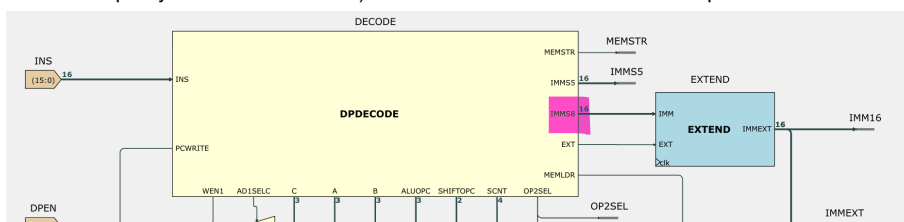
For each register file data output port (for example DOUT2, DOUT3):

- state which address field in the instruction word (a, b, or c) controls which register is read on that port;
- state which ALU input port this register file output is connected to.



A inputs to AD2 and controls Ra & b inputs to AD3 and controls Rb

2. Immediate value into the ALU. Name the ALU input port through which immediate values enter (this is the same port you used in Section 2) and confirm how it is connected to dpdecode.

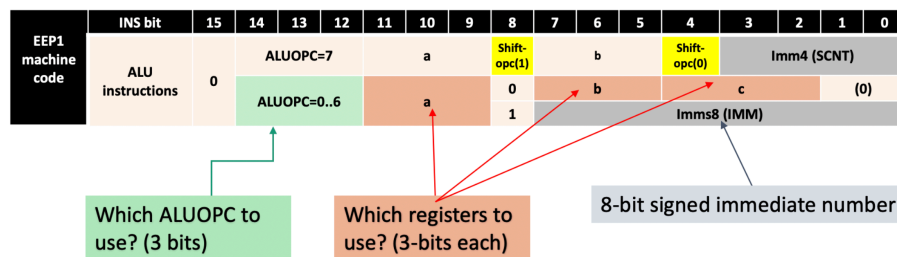


A	B	C	IMM	OP2SEL	AD1SEL	REGFILE.Din1	REGFILE.AD1
a	b	c	n	0	0	b	a
a	b	c	n	0	1	b	c

[illegible]

Extension questions

1. When executing a MOV Rn, #IMM instruction, how does the register read port of the register file behave? Does it still output register data? Explain what happens to this data and which hardware arrangement ensures that the immediate value, rather than the register output, is written into the destination register.



When it is #IMM, the 8th bit, which connects to OP2SEL = 1, so REGFILE.Din1 always imports IMM, register output won't be chosen.

2. Explain why having only a single read port from the register file to the ALU would severely limit the CPU's ability to perform useful arithmetic operations in a single cycle.

With only one read port, the CPU cannot read two source registers in the same cycle. Since most arithmetic operations require two operands, this would force multi-cycle execution and significantly reduce performance.

3. Write a new text file containing the four lines of assembly code in Figure 7. Add three more lines of code that add the contents of registers R0–R3 and store the result in R4, without modifying registers R5–R7. Simulate your code in `eeplab1` and use the waveform viewer to confirm that the result in R4 is correct.

```
MOV R0, #12
MOV R1, #-7
MOV R2, #233
MOV R3, #1
```

We add:

```
ADD R4, R0, R1
ADD R4, R4, R2
ADD R4, R4, R3
```

	0	5	10	15	20	25	30	0
Reg16x8.R4(15:0)	x0000	xFFF					xFFF	x0000

So R4 will be $12 - 7 + 233 + 1$ in signed 8 bits.
 233 is overflow and it actually = $233 - 256 = -23$
 Now $R4 = 12 - 7 - 23 + 1 = -17$
 = 1111 1111 1110 1111 = 0xFFEF,
 which is correct.